

Draw It or Lose It.

CS 230 Project Software Design Template

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2

Document Revision History 2

Executive Summary 3

Requirements 3

Design Constraints 3

System Architecture View 3

Domain Model 3

Evaluation 4

Recommendations 5

Document Revision History

Version	Date	Author	Comments
1.0	09/17/2026	Christopher Walker	Completed Project One software design sections for The Gaming Room.
1.1	10/01/2026	Christopher Walker	Completed Project Two platform evaluation for The Gaming Room.

1.0	10/06/2026	Christopher Walker	Completed Project Three recommendations for operating platform, architecture, storage, memory, distributed systems, networking, and security.
-----	------------	--------------------	---

Instructions

Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The Gaming Room has reached out to Creative Technology Solutions (CTS) to improve its existing Android game Draw It or Lose It becomes a multi-platform app that will be compatible with multiple devices and operating systems. The game is about teams guessing pictures, which come out bit by little as clues. The program should be object-oriented to clearly segregate the games, teams and players, and to avoid duplicate names and unnecessary service instances as the application grows. The proposed approach uses a shared Entity base class for common information, Inheritance for Game, Team and Player classes, Singleton pattern for GameService and iteration across collections when checking for existent names. This approach gives The Gaming Room a durable foundation to construct a web version of the game and makes it easy to support more platforms in the future.

Requirements

The Draw It or Lose It web version must support one or more teams each game for the client, with many players on each team. Game and team names must be unique to enable users to know if a name is taken before establishing a new one. Games also need unique identification as do teams and players. There should be only one instance of GameService object in memory at any given moment. The program needs to be designed such that these requirements may be continuously enforced as the game is developed beyond its initial Android only setting.

Design Constraints

The new application has a number of constraints which have an impact on the design. The game is being transferred from an Android only application to a web based distributed environment therefore the design should function across several operating systems, browsers and device types rather than be relying on one platform. The application also needs a strong mechanism of exchanging data so that game and team names are unique and each game, team and player is given a unique identity. Why singleton? There should only be one GameService object in memory and singleton design is appropriate for controlling access to the service. The design also

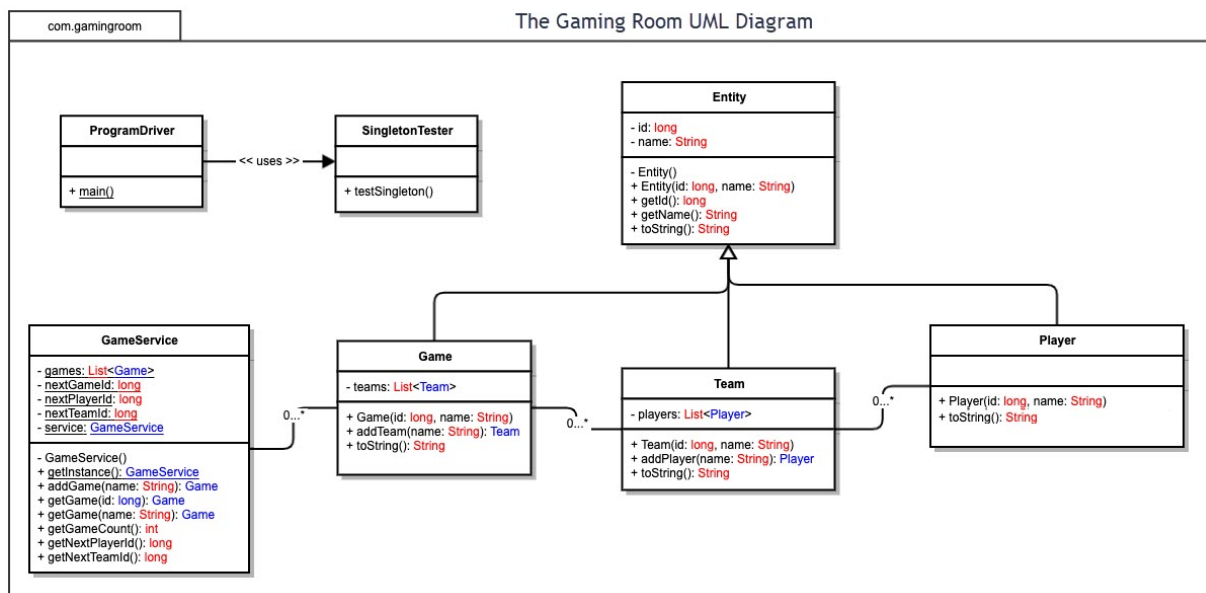
has to retain the relation between games, teams and players and be easy to maintain and extend. This forces you to use reusable classes and to build your software with concepts that reduce repeated code and help your program behave predictably as it grows.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The UML class diagram displays the interaction between the key classes in the program. Entity is the foundation class. It has the common id and name attributes. Game, Team and Player inherit from Entity. This shows inheritance and avoids code duplication. The client wants a game to have several teams and a team to have many players. So a Game has a collection of Team objects and each Team has a collection of Player objects. GameService handles the collection of games and provides methods to add and find games. It follows the Singleton design pattern thus there will only ever be one instance of GameService in memory. It also shows encapsulation as the class data is encapsulated within the objects and accessed by constructors and methods. Iteration is used when searching collections before games, teams or players are added to assist prevent duplicate names. Together, these concepts and design patterns of object oriented programming form a clear framework that efficiently serves the client's objectives.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	macOS can run web server software and Java-based applications, so it can host Draw It or Lose It. However, current macOS is mainly a desktop platform rather than a dedicated enterprise server OS. Production hosting would also require Apple hardware, which can increase hardware cost. It is better suited to development and testing than large-scale hosting.	Linux is well suited for hosting a scalable web application. Distributions such as Ubuntu Server can run Java, web servers, databases, containers, and cloud services needed for thousands of users. Many Linux distributions have no operating-system license fee, although paid support is available. Its stability, security, and low licensing cost make it attractive for production hosting.	Windows Server provides a supported server deployment environment with IIS, Java support, databases, virtualization, and Microsoft cloud integration. It can scale for a web application, but Windows Server uses commercial licensing, commonly based on processor cores, and some configurations may require additional access licenses. This can make it more expensive than Linux.	Android and iOS devices should operate as clients rather than host the main game server. The back-end application should run on a dedicated server or cloud platform, while phones and tablets connect to it over the internet. Keeping game logic and shared data on the server improves consistency, scalability, security, and maintenance.

Client Side	Mac users can access the game through a responsive HTML interface in Safari, Chrome, Firefox, or Edge. Developers should test Safari and different display sizes to confirm that page layout, images, controls, authentication, and network communication behave correctly. A browser-based design avoids creating a separate macOS desktop application.	Linux clients can use modern browsers such as Chrome and Firefox to access the same responsive web interface. Development should rely on standards-based HTML, CSS, and JavaScript and should include cross-browser testing. This reduces platform-specific code while still requiring testing of browser behavior, screen sizes, and performance on Linux systems.	Windows users can run the game in Edge, Chrome, or Firefox. The team should test the responsive interface across common browsers and screen resolutions and verify controls, images, game timing, and communication with the server. Using a web client reduces the time and cost of maintaining a separate Windows application.	The interface must adapt to Android and iOS phones and tablets. Responsive HTML, CSS, and JavaScript should support different screen sizes, touch input, portrait and landscape orientation, and slower or changing network connections. Testing should include Safari on iOS and Chrome on Android. A shared web interface lowers cost compared with maintaining two separate native apps.
---------------------------	---	--	---	--

Development Tools	macOS supports Java, HTML, CSS, JavaScript, Git, IntelliJ IDEA, Eclipse, Visual Studio Code, Maven, and Gradle. Xcode is also available for iOS-specific testing or future native development . Many tools have free editions, but the team needs Mac hardware for reliable Safari and iOS testing, and Apple developer services may add cost if native apps are distributed.	Linux supports Java, JavaScript, HTML, CSS, Git, IntelliJ IDEA, Eclipse, Visual Studio Code, Maven, Gradle, Docker, and many command-line tools. Many are open source or available at no cost, which reduces development expense. Team members need Linux and server-administration knowledge, but a separate Linux-only development team is not necessary if common web technologies are used.	Windows supports Java, HTML, CSS, JavaScript, Visual Studio Code, Visual Studio, IntelliJ IDEA, Eclipse, Git, Maven, Gradle, and Microsoft development services. Free editions are available for many tools, while some professional or enterprise products require licenses. Developers with Microsoft or Windows Server experience may be useful, but the shared web codebase limits the need for a separate Windows team.	Android Studio supports Android development and testing, while Xcode is used for Apple-specific development and iOS simulation. The shared web application can still be created with HTML, CSS, JavaScript, Java, Git, and browser developer tools. The team needs mobile testing expertise, especially for touch interfaces and device sizes, but responsive web design reduces the need for completely separate Android and iOS teams.
---------------------------------	--	--	---	---

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

Operating Platform: I recommend using Linux, specifically an Ubuntu Server Long-Term

Support (LTS) distribution, as the server operating platform for Draw It or Lose It. Linux is well suited for web-based and distributed applications because it is stable, secure, scalable, and widely supported by cloud-hosting providers. It also has no per-server licensing cost and supports Java and the Dropwizard framework used by the application. By placing the game logic on a Linux server and delivering the client through a web browser or platform-independent application interface, The Gaming Room can support users on Windows, macOS, Android, iOS, and other systems without maintaining a separate server implementation for each platform.

Operating Systems Architectures: Linux uses a monolithic kernel architecture with modular components that can be loaded as needed. The kernel manages processes, memory, file systems, device drivers, security permissions, and network communication while user applications run in protected user space. Ubuntu Server supports 64-bit hardware, multiprocessing, multitasking, and multi-user operation, which are important for hosting several Draw It or Lose It game sessions at the same time. Linux also provides a virtual file system layer that allows applications to work consistently with different storage devices and file-system types. For this application, the Java Virtual Machine runs above the operating system and provides another layer of portability, allowing the same Java server code to operate on compatible Linux hardware without being tied to one physical server configuration.

Storage Management: For server storage, I recommend the ext4 file system on the Linux server together with a relational database such as PostgreSQL for persistent game and user data. ext4 is a mature Linux file system that supports journaling, permissions, large files, and reliable recovery after an unexpected shutdown. The file system can store application files, logs, configuration files, and image assets, while PostgreSQL can organize structured data such as user accounts, teams, game sessions, scores, and other records that must remain available after a process restarts. Regular backups and database replication can be added so that data can be restored if storage hardware fails or a server becomes unavailable.

Memory Management: Linux uses virtual memory, paging, demand paging, and process isolation to manage physical memory efficiently. Each Draw It or Lose It server process receives its own protected virtual address space, which reduces the chance that one process will corrupt the memory used by another. Frequently used data can remain in memory for fast access, while less active memory pages may be moved or reclaimed when the system experiences pressure. Linux also uses a page cache to improve access to files and application resources. Because the server is written in Java, the Java Virtual Machine adds automatic garbage collection to release objects that are no longer needed. Together, Linux memory management and JVM garbage collection allow multiple games and user sessions to run concurrently while helping the server use available RAM efficiently.

Distributed Systems and Networks: Draw It or Lose It should use a client-server distributed architecture. The central Linux server would host the game services and shared data, while clients on Windows, macOS, Android, iOS, or other platforms communicate with the server through a web browser or application using RESTful web services. Communication should use standard TCP/IP networking with HTTPS so that clients can connect over local networks or the

internet without depending on a specific operating system. A load balancer can distribute requests across multiple application servers as usage grows, and the database can be protected with replication and backups. Because distributed systems depend on network connectivity and multiple components, the software should include timeouts, retry logic, clear error messages, session recovery, health checks, and logging. Redundant servers and network connections can also reduce downtime if a server, service, or connection fails.

Security: Security should be applied at the operating-system, application, and network levels. All communication between clients and the server should use HTTPS with current TLS encryption so that login information and game data are protected while traveling across the network. User passwords should never be stored as plain text; they should be salted and hashed with a strong password-hashing algorithm. The application should use authentication and role-based authorization so users can access only the functions and data appropriate to their accounts. On the Linux server, The Gaming Room should apply the principle of least privilege, restrict administrative access, use firewall rules, install security updates promptly, and disable unnecessary services. Sensitive stored data should be encrypted when appropriate, and regular backups, audit logs, input validation, and monitoring should be used to detect misuse and support recovery. These controls provide consistent protection for users even when they connect from different operating systems and device types.